Suite Contrôleur de ressources (CRUD)Partie 4

Route resource

🗀 **Fichier : routes/web.php**

```
use App\Http\Controllers\UserController;


Route::resource('users', UserController::class);
```

Explication

- Route::resource() est une **méthode spéciale** de Laravel
- Elle permet de créer **plusieurs routes automatiquement**
- On évite d'écrire 7 routes à la main

Cette **seule ligne** crée automatiquement **7 routes**

URL	Méthode HTTP	Rôle	Méthode appelée
/users	GET	Afficher la liste	index
/users/create	GET	Afficher formulaire	create
/users	POST	Ajouter données	store
/users/{id}	GET	Afficher 1 élément	show
/users/{id}/edit	GET	Formulaire modification	edit
/users/{id}	PUT	Modifier	update
/users/{id}	DELETE	Supprimer	destroy

Une seule ligne = 7 routes prêtes à l'emploi

Contrôleur avec données simulées (array)

🗀 **Fichier : app/Http/Controllers/UserController.php**

```
namespace App\Http\Controllers;
use Illuminate\Http\Request;
class UserController extends Controller
{
   // Données simulées (FAKE DB)
   private $users = [
      ['id' => 1, 'name' => 'Ali', 'email' => 'ali@test.com'],
      ['id' => 2, 'name' => 'Sara', 'email' => 'sara@test.com'],
   ];
```

Explication

- Normalement, les données viennent de la **base de données**
- Ici, on **simule une base de données** avec un tableau PHP
- $users représente une **liste d'utilisateurs**

Pourquoi on fait ça ?

Pour comprendre **le CRUD sans complexité**

Méthode index()

Route : GET /users

```
public function index()
{
   $users = $this->users;
   return view('users.index', compact('users'));
```

```
}
```

Explication

- index() sert toujours à **afficher une liste**
- $this->users → on récupère les données simulées
- view('users.index') → on appelle la vue
- compact('users') → on envoie les données à la vue

Schéma à expliquer :

Navigateur → Route → Controller → View

Méthode create()

Route : GET /users/create

```
public function create()
{
   return view('users.create');
}
```

Explication

- Cette méthode **n'enregistre rien**
- Elle affiche juste un **formulaire**
- C'est la page où l'utilisateur **remplit les champs**

Règle importante :

GET = afficher, POST = envoyer

Méthode store()

Route : POST /users

```
public function store(Request $request)
```

```
{
    $newUser = [
        'id' => count($this->users) + 1,
        'name' => $request->name,
        'email' => $request->email,
    ];

    return view('users.store', compact('newUser'));
}
```

Explication

- Request $request contient **les données du formulaire**
- $request->name → valeur saisie dans l'input name
- On crée un **nouvel utilisateur (simulation)**
- On ne sauvegarde pas encore, on **affiche seulement**

----Plus tard, cette méthode enregistrera dans la base de données

Vue index (liste des utilisateurs)

🗀 **resources/views/users/index.blade.php**

```
<h2>Liste des utilisateurs</h2>
<a href="/users/create">➕ Ajouter utilisateur</a>
<ul>
@foreach($users as $user)
    <li>{{ $user['name'] }} - {{ $user['email'] }}</li>
@endforeach
</ul>
```

Explication

- $users vient du contrôleur
- @foreach permet de **boucler sur les données**
- {{ }} sert à **afficher une variable**
- Le lien permet d'aller au **formulaire**

Notion clé :

La vue **ne contient pas de logique**, juste de l'affichage

Vue create (formulaire)

resources/views/users/create.blade.php

```
<h2>Créer utilisateur</h2>
<form method="POST" action="/users">
   @csrf
   <input type="text" name="name" placeholder="Nom"><br><br>
   <input type="email" name="email" placeholder="Email"><br><br>

   <button type="submit">Enregistrer</button>
</form>
```

Explication

- method="POST" → envoi de données
- action="/users" → route store
- @csrf → sécurité obligatoire Laravel
- name="name" → utilisé par $request->name

Erreur fréquente à expliquer :

Sans @csrf, Laravel bloque le formulaire

Vue store (résultat)

🗀 **resources/views/users/store.blade.php**

```
<h2>Utilisateur créé (simulation)</h2>


<p>Nom : {{ $newUser['name'] }}</p>
<p>Email : {{ $newUser['email'] }}</p>


<a href="/users">Retour à la liste</a>
```

Explication

- Cette page montre **les données reçues**
- Elle confirme que le formulaire fonctionne
- Le lien permet de revenir à la liste

« Les données ont circulé correctement »